

Full authentication with Next.js (using cookies) and Django (JWT)

Overview

This guide shows a secure, production-ready approach where Django issues JWT access and refresh tokens, and the Next.js frontend uses HttpOnly cookies to store refresh tokens (and optionally access tokens). Key points:

- Use HttpOnly, Secure, SameSite cookies to store tokens (prevents XSS theft).
- Store long-lived refresh token in HttpOnly cookie; keep short-lived access token in memory or also HttpOnly.
- Use a refresh endpoint on the backend to rotate refresh tokens.
- For cross-origin setups enable CORS and use credentials with fetch/axios.
- Server-side rendered pages (getServerSideProps) should forward incoming cookies to the backend for auth.

Django (backend) - Recommended endpoints

- 1) POST /api/auth/login/ -> Accepts credentials, returns tokens and sets cookies.
- 2) POST /api/auth/refresh/ -> Uses refresh cookie to issue a new access token (rotates refresh).
- 3) POST /api/auth/logout/ -> Revokes refresh token and clears cookies.
- 4) GET /api/auth/me/ -> Returns current user (checks access token from Authorization header or cookie).

Django example (using django-rest-framework-simplejwt)

settings.py (relevant)

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=5),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
}

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    ...
}
```

```
# views.py (simplified)
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework_simplejwt.tokens import RefreshToken
from django.conf import settings
```

```
def set_auth_cookies(response, access_token, refresh_token):
    # Access token as cookie (optional) and refresh token in HttpOnly cookie
    response.set_cookie(
        key='refresh', value=refresh_token,
        httponly=True, secure=not settings.DEBUG, samesite='Lax', path='/api/auth/'
    )
    response.set_cookie(
        key='access', value=access_token,
        httponly=False, secure=not settings.DEBUG, samesite='Lax', path='/'
    )
    return response
```

```

class LoginView(APIView):
    def post(self, request):
        # validate credentials, get user
        # create tokens
        refresh = RefreshToken.for_user(user)
        access = str(refresh.access_token)
        refresh_token = str(refresh)
        res = Response({"detail": "logged in"}, status=status.HTTP_200_OK)
        set_auth_cookies(res, access, refresh_token)
        return res

class RefreshView(APIView):
    def post(self, request):
        # read refresh token from cookie
        refresh_token = request.COOKIES.get('refresh')
        if not refresh_token:
            return Response(status=401)
        try:
            refresh = RefreshToken(refresh_token)
            new_access = str(refresh.access_token)
            if settings.SIMPLE_JWT['ROTATE_REFRESH_TOKENS']:
                new_refresh = refresh.rotate()
                refresh_token = str(new_refresh)
            res = Response({"access": new_access})
            set_auth_cookies(res, new_access, refresh_token)
            return res
        except Exception:
            return Response(status=401)

class LogoutView(APIView):
    def post(self, request):
        # optionally blacklist refresh token
        res = Response({"detail": "logged out"})
        res.delete_cookie('refresh', path='/api/auth/')
        res.delete_cookie('access', path='/')
        return res

```

Notes:

- Keep access token short-lived (e.g., 5 minutes).
- Use ROTATE_REFRESH_TOKENS + BLACKLIST_AFTER_ROTATION to mitigate stolen refresh token risks.
- You can keep access token in a non-HttpOnly cookie or only in memory. Storing access token in HttpOnly cookie simplifies SSR but requires backend to read cookies.

Cross-origin (frontend on different domain)

```

- In Django CORS settings allow credentials:
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOWED_ORIGINS = ['https://your-nextjs-domain.com']
- When calling backend from browser include credentials:
fetch('https://api.example.com/api/auth/login/', {
  method: 'POST',
  credentials: 'include',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({username, password})
});

```

Next.js (frontend) patterns

1) Authentication flow (recommended)

- Client login POSTs to Next.js API route (/api/auth/login) which proxies to Django, so browser sees same-origin cookie setting.
- Next.js API route forwards credentials to Django, receives Set-Cookie headers, then returns a response to the browser WITHOUT exposing tokens.

2) Using fetch/axios from browser directly to Django:

- Allowed but you'll need CORS and `credentials: 'include'`.
- It's better to proxy via Next.js API routes for same-origin cookies and simplified CSRF.

Next.js examples

```
# pages/api/auth/login.js
export default async function handler(req, res) {
  if (req.method !== 'POST') return res.status(405).end();
  const { username, password } = req.body;
  // Forward to Django
  const response = await fetch(process.env.API_URL + '/api/auth/login/', {
    method: 'POST',
    credentials: 'include',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({ username, password }),
  });
  const text = await response.text();
  res.status(response.status).send(text);
}

# pages/api/auth/refresh.js
export default async function handler(req, res) {
  // Forward refresh request; cookies from browser are forwarded by Next's fetch automatically?
  // In API route, you must forward incoming cookies header:
  const response = await fetch(process.env.API_URL + '/api/auth/refresh/', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Cookie': req.headers.cookie || '',
    },
  });
  const data = await response.json();
  // forward set-cookie headers if present
  const sc = response.headers.get('set-cookie');
  if (sc) res.setHeader('Set-Cookie', sc);
  res.status(response.status).json(data);
}

# Client-side login (React)
async function login(username, password) {
  const res = await fetch('/api/auth/login', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({ username, password }),
    credentials: 'include'
  });
  return res.ok;
}
```

Server-side rendering and protected pages

```
# pages/dashboard.js
export async function getServerSideProps(context) {
  // Forward cookie to Django user endpoint
  const cookie = context.req.headers.cookie || '';
  const apiRes = await fetch(process.env.API_URL + '/api/auth/me/', {
    headers: {'Cookie': cookie },
  });
  if (apiRes.status !== 200) {
    return { redirect: { destination: '/login', permanent: false } };
  }
  const user = await apiRes.json();
  return { props: { user } };
}
```

}

Token refresh strategy

- On the client, keep an in-memory copy of access token if you use it for Authorization header.
- When 401 occurs, call `/api/auth/refresh` to get a new access token (cookies propagate) and retry the original request.
- Alternatively, use silent background refresh on window focus or setInterval slightly before access token expiry.

Security recommendations

- ALWAYS use HttpOnly + Secure cookies on production (Secure requires HTTPS).
- Set SameSite=Lax or Strict for login cookies to reduce CSRF risk.
- Implement CSRF protection for state-changing requests if access token is stored in non-HttpOnly cookie or if you accept cookies cross-site.
- Prefer rotating refresh tokens and blacklist on logout.
- Limit cookie path (e.g., refresh cookie path='/api/auth/') to reduce exposure.
- Rate-limit auth endpoints and enforce strong password policies.

Putting it together - checklist

- [] Django: SIMPLE_JWT configured with short access, rotated refresh.
- [] Django: login/refresh/logout endpoints that set HttpOnly cookies.
- [] Django: CORS_ALLOW_CREDENTIALS=True and allowed origins set.
- [] Next.js: use API routes to proxy auth calls (same-origin), use credentials: 'include'.
- [] Next.js: SSR pages forward req.headers.cookie to backend.
- [] Implement token refresh handling and logout flow.

References & further reading

- djangorestframework-simplejwt docs
- OWASP recommendations on JWT and cookies
- Next.js docs: API routes and getServerSideProps